Simulador de cinemática 3D para sistemas robóticos de 2 eslabones

Emmanuel Morales Loaiza

Facultad de Ingeniería, Instituto Tecnológico Metropolitano

1 Resumen

Este proyecto presenta el diseño y desarrollo de un software de simulación interactiva para la cinemática de un brazo acrobático con dos eslabones y tres grados de libertad; utilizando Python y librerías de calculo científico como NumPy y matplotlib,se implementó un modelo matemático basado en transformaciones trigonométricas para la representación visual en un entorno 3D. El software integra una interfaz de usuario dinámica mediante controladores deslizantes (sliders) que permiten la manipulación de variables angulares y la actualización de coordenadas en tiempo real.Donde finalmente la herramienta fue empaquetada para crear el archivo ejecutable.

2 Introducción

La robótica, no es un tema actual, viene desde hace décadas trabajando cada vez mas por expandirse; y así como se expande se busca que su comprensión sea algo simple para todo el mundo.

Lo que sucede es que la robótica no es un tema de "prendamos haber si funciona",todo lo contrario, requiere planeamiento,análisis, desarrollo, entre muchos otros factores que pueden que pueden entorpecer la creación de algún producto, por eso la simulación es tan importante; porque evita que ocurran incidentes de carácter grotesco.

Uno de los factores en los que mas fallos suelen encontrarse, primordialmente en sistemas de robótica, es la cinemática aplicada, ver como se mueve un sistema y saber que variables corregir a tiempo, puede exonerar a las personas de perder tiempo, recursos, entre otros recursos que en el mundo de la robótica implican grandes costos.

Por eso y muchas otras cosas mas, es que idee el desarrollo de este software, para poder ver como se mueven las variables en la simulación, para que el día de mañana la construcción de estos sistemas sea más simple.

3 Marco teórico

El desarrollo de el simulador de basa en cinemática directa, lo cual permite determinar de manera simple la posición de el brazo en un espacio tridimensional.

3.1 paramentos del sistema

Para el modelo de 2 eslabones y 3 grados de libertad, se deben tener en cuenta los siguientes parámetros:

-L1 y L2: longitud de cada uno de los eslabones.

-θ:Angulo de rotación de la base

-ϕ:Angulo de elevación de el primer eslabón

-β:Angulo de el segundo eslabón

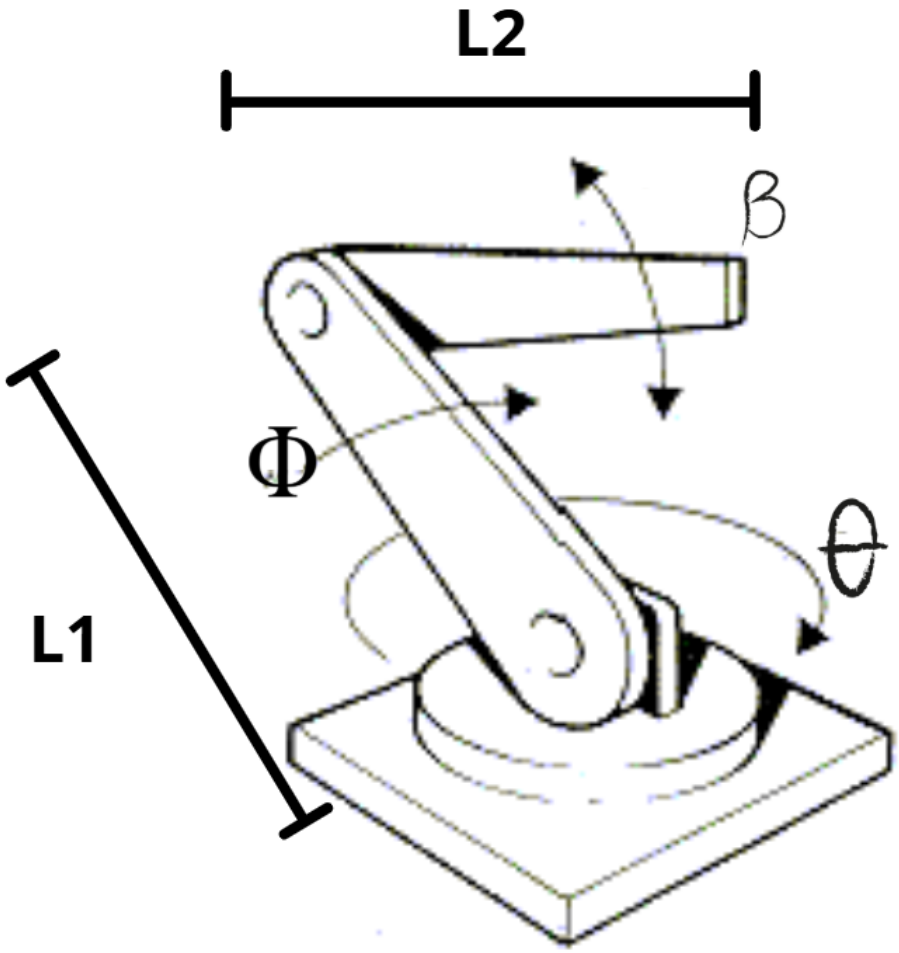

Figure 1: tomado de: https://inteligenciartificiallmca.wordpress.com/2017/05/10/2-5-grados-de-libertad-de-un-robot/

3.2 deducción de coordenadas

La posición de cada articulación se calcula mediante la descomposición de vectores utilizando funciones trigonométricas en coordenadas esféricas y polares. Posición del codo: representa el final del primer eslabón.

$$X_1 = L_1 \cdot cos(\phi) \cdot cos(\theta)$$

$$Y_1 = L_1 \cdot cos(\phi) \cdot sen(\theta)$$

$$Z_1 = L_1 \cdot sin(\phi)$$

Posición de la muñeca: representa el final de el segundo eslabón:

$$X_2 = X_1 + L_2 \cdot cos(\phi + \beta) \cdot cos(\theta)$$

$$Y_2 = Y_1 + L_2 \cdot cos(\phi + \beta) \cdot sen(\theta)$$

$$Z_2 = Z_1 + L_2 \cdot sin(\phi + \beta)$$

3.3 Representación en el espacio

El modelo utiliza el sistema coordenado de mano derecha.lo que hace el software es tomar estos valores hacer arreglos vectoriales para hacer el renderizado dinámico.

4 Metodología y desarrollo de software

Primordialmente antes de hacer el sistema, lo idóneo es ver que clase de cosas necesito ejecutar, en esta caso es simple, como las funciones que me dan la posición de el primer y el segundo eslabón son básicamente arreglos matriciales y si ademas de eso quiero grafique que es lo que estoy haciendo, necesito algo que me permita visualizar en 3D que es lo que tengo armado; por lo tanto concluí que las mejores opciones fueron NumPy para los arreglos matriciales y Matplotlib para el renderizado de el sistema.A continuación, se explica el paso a paso de que es lo que hace cada linea de el código, para que no quede en el aire, que hace cada cosa, por si alguien desea implementar mejoras.

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib.widgets import Slider

fig = plt.figure(figsize=(8, 8))
ax = fig.add_subplot(111, projection='3d')

L1, L2 = 1.5, 1.0
theta_init, phi_init, beta_init = 45, 45, 45
```

En esta parte, simplemente se dan los aspectos iniciales, se importan como se menciono anteriormente las librearías a usar.

Paso seguido se crea un objeto muy especial, el cual es la figura sobre la que vamos a trabajar, se le da un ancho y un largo; y se crea "una vertiente" de lo que vendría hacer un grafica normal, lo que sucede es que posteriormente se agregara a la figura.

Se instancia los valores iniciales para la longitud de los eslabones y los de los ángulos, por eso cada que se cierra el programa y se vuele a ejecutar el brazo regresa a su posición de origen

```python
def calcular_puntos(t, p, b):  2 usages
    t_rad, p_rad, b_rad = np.radians(t), np.radians(p), np.radians(b)
    #Codo
    x1 = L1 * np.cos(p_rad) * np.cos(t_rad)
    y1 = L1 * np.cos(p_rad) * np.sin(t_rad)
    z1 = L1 * np.sin(p_rad)

    #Muñeca
    x2 = x1 + L2 * np.cos(p_rad + b_rad) * np.cos(t_rad)
    y2 = y1 + L2 * np.cos(p_rad + b_rad) * np.sin(t_rad)
    z2 = z1 + L2 * np.sin(p_rad + b_rad)

    return x1, y1, z1, x2, y2, z2
```

Aquí se usan las ecuaciones presentadas anteriormente, se crea la función en la cual se le pasan los parámetros, θ,ϕ,β que se pasan a radianes para hacer los cálculos correctamente, lo cual lo podemos traducir de la siguiente manera:

$$\begin{pmatrix} X_1 \\ Y_1 \\ Z_1 \end{pmatrix} = L_1 \begin{pmatrix} cos(\phi) \cdot cos(\theta) \\ cos(\phi) \cdot sen(\theta) \\ sin(\phi) \end{pmatrix}$$

y en la segunda parte de la función se calculan de una vez los parámetros para la muñeca:

$$\begin{pmatrix} X_2 \\ Y_2 \\ Z_2 \end{pmatrix} = \begin{pmatrix} X_1 \\ Y_1 \\ Z_1 \end{pmatrix} + L_2 \begin{pmatrix} cos(\phi + \beta) \cdot cos(\theta) \\ cos(\phi + \beta) \cdot sen(\theta) \\ sin(\phi + \beta) \end{pmatrix}$$

y finalmente se retornan los valores calculados.

```
#puntos iniciales
x1i, y1i, z1i, x2i, y2i, z2i = calcular_puntos(theta_init, phi_init, beta_init)

#Eslabones
line1, = ax.plot( xs: [0, x1i], ys: [0, y1i], *args: [0, z1i],
                  '-', linewidth=6, color='blue', label='Eslabón 1', zorder=1)
line2, = ax.plot( xs: [x1i, x2i], ys: [y1i, y2i], *args: [z1i, z2i],
                  '-', linewidth=6, color='red', label='Eslabón 2', zorder=1)

#articulaciones
ax.scatter( xs: [0], ys: [0], zs: [0], color='black', s=150, label='Articulaciones', zorder=5)  # Base fija
codo_plot = ax.scatter( xs: [x1i], ys: [y1i], zs: [z1i], color='black', s=150, edgecolor='black', zorder=10)
muneca_plot = ax.scatter( xs: [x2i], ys: [y2i], zs: [z2i], color='black', s=100, edgecolor='black', zorder=10)

ax.legend()
plt.subplots_adjust(bottom=0.3)

ax_theta = plt.axes([0.25, 0.18, 0.55, 0.03])
ax_phi = plt.axes([0.25, 0.13, 0.55, 0.03])
ax_beta = plt.axes([0.25, 0.08, 0.55, 0.03])

theta_slider = Slider(ax_theta, label: 'Theta (Base)', valmin: 0, valmax: 180, valinit=theta_init)
phi_slider = Slider(ax_phi, label: 'Phi (Hombro)', valmin: 0, valmax: 180, valinit=phi_init)
beta_slider = Slider(ax_beta, label: 'Beta (Codo) ', -90, valmax: 90, valinit=beta_init)
```

para simplificar la cuestión y tener mas conciencia de que es lo que se esta calculando se pasan los valores calculados a nuevas variables, para evitar errores; posteriormente se grafica como mencione anteriormente en la posición inicial.

se grafica cada eslabón, usando como puntos de referencia las nuevas variables a las cuales les dimos los valores de lo calculado, esto se hace así,porque, posteriormente se requiere usar de nuevo esta función para calcular los valores modificados.

Se grafica cada articulación que permite el movimiento, como se puede observar el punto coordenado (0,0,0) es el origen de el movimiento es la base en la que se sujeta el brazo, y las otras dos articulaciones es en donde termina ”la punta” de el eslabón.

Después se instancia y se grafican tres objetos, que serán los sliders que nos van a permitir, variar los ángulos.

```python
def act(val):
    t = theta_slider.val
    p = phi_slider.val
    b = beta_slider.val

    x1, y1, z1, x2, y2, z2 = calcular_puntos(t, p, b)

    line1.set_data([0, x1], [0, y1])
    line1.set_3d_properties([0, z1])

    line2.set_data([x1, x2], [y1, y2])
    line2.set_3d_properties([z1, z2])

    codo_plot._offsets3d = ([x1], [y1], [z1])
    muneca_plot._offsets3d = ([x2], [y2], [z2])

    fig.canvas.draw_idle()
```

En esta parte se crea la función de activación, la cual permite recalcular los valores que ya teníamos al aplicar la variación de parámetros y como mencione anteriormente, se vuelve a llamar a la función inicial para calcular los puntos, después se le dan los nuevos valores a los dos eslabones y básicamente se le dice a la figura creada que tome estos nuevos datos.

```
theta_slider.on_changed(act)
phi_slider.on_changed(act)
beta_slider.on_changed(act)

ax.set_xlim([-2.5, 2.5])
ax.set_ylim([-2.5, 2.5])
ax.set_zlim([0, 3])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

plt.show()
```

Finalmente se actualizan los valores, se le dan restricciones al modelo para que no se vuelva loco, se da nombre a los ejes y se grafica finalmente todo lo recorrido.

5 Implementación y resultados

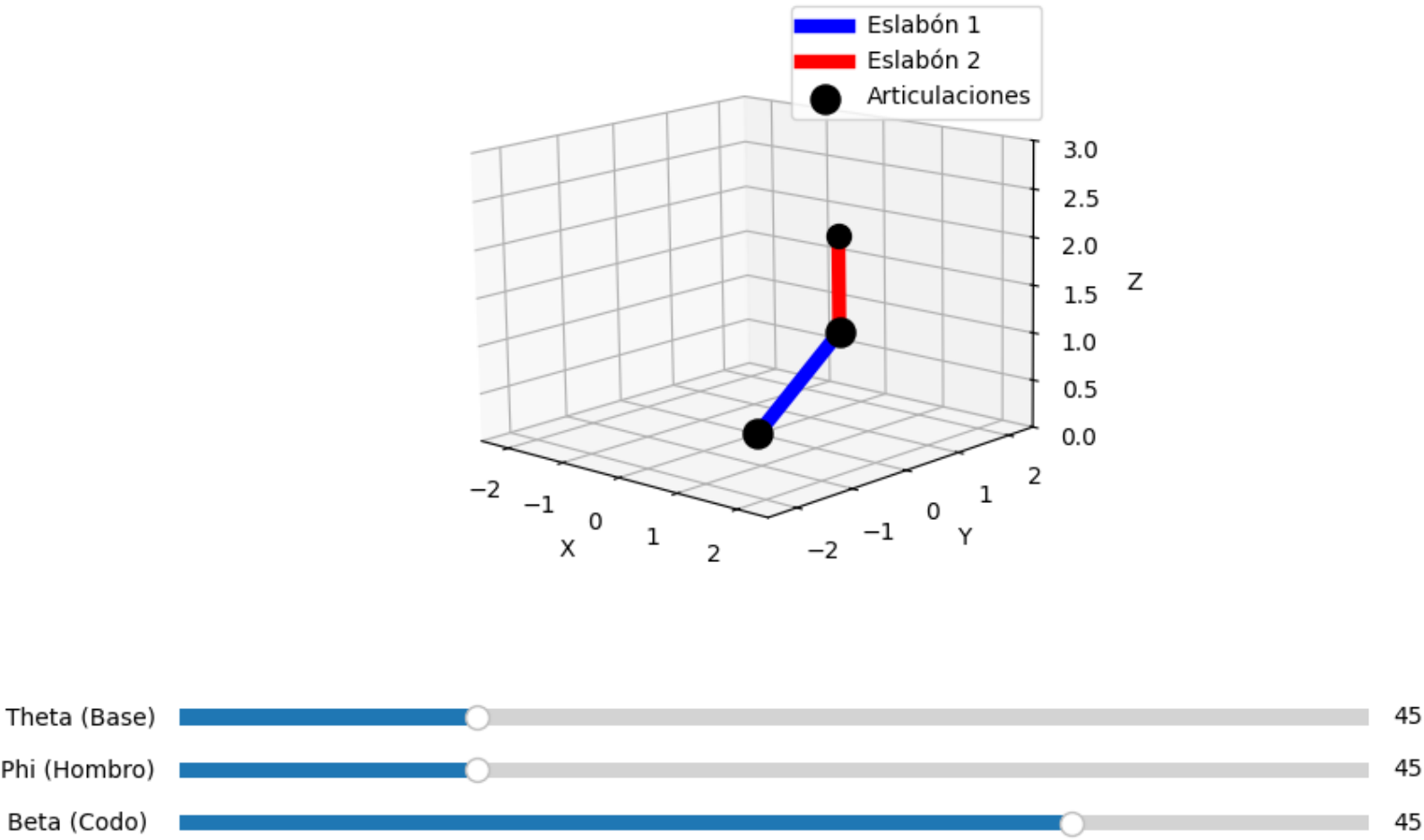

Figure 2: Se observa que al ejecutar el programa los ángulos son los designados por las variables principales primarias, que son las que dan la instancia de que cada angulo debe ser de 45 grados.

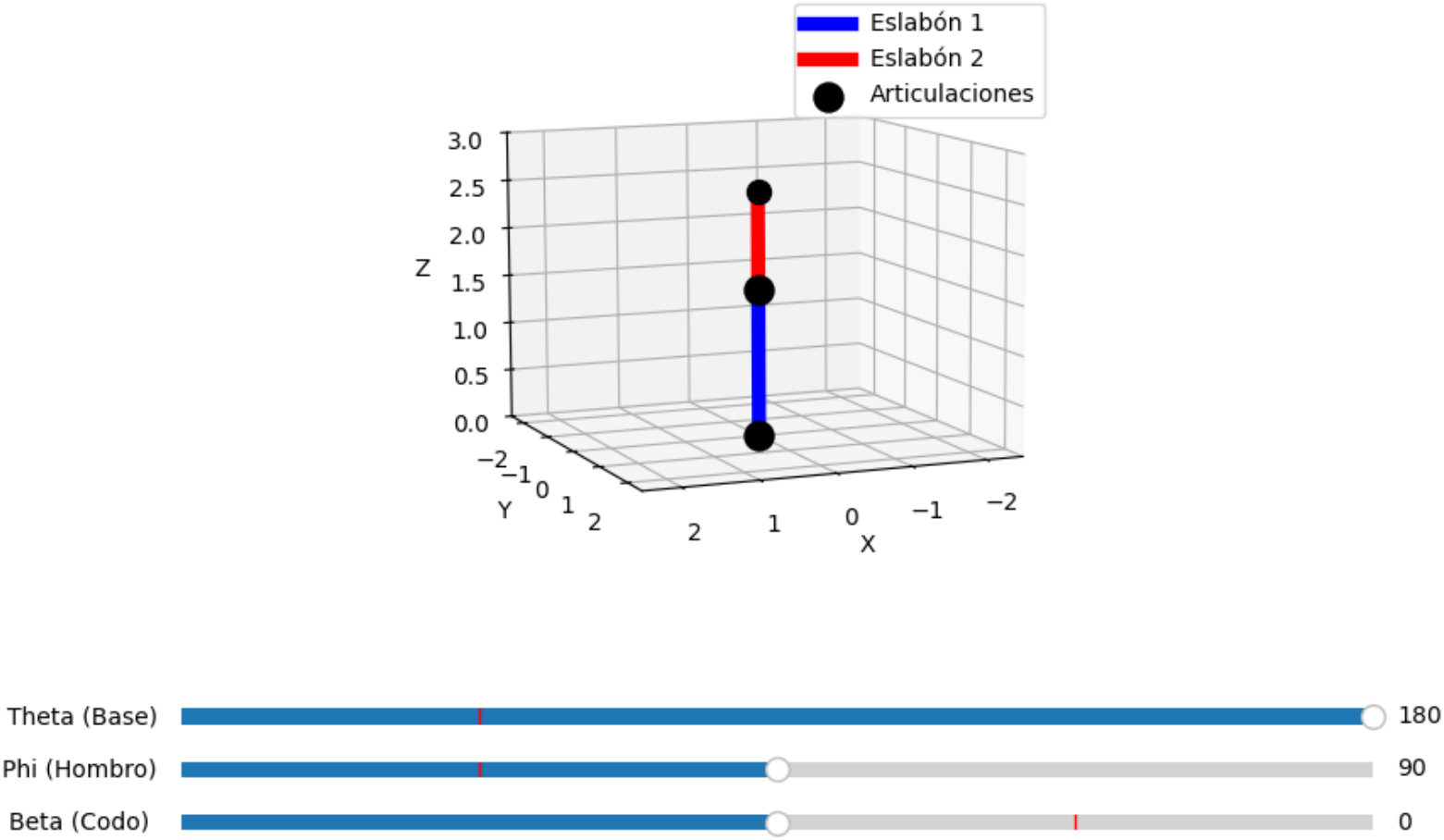

Figure 3: Aquí se puede observar al brazo completamente recto, el codo y el hombro están en una escala respectiva que les permite esta posición.

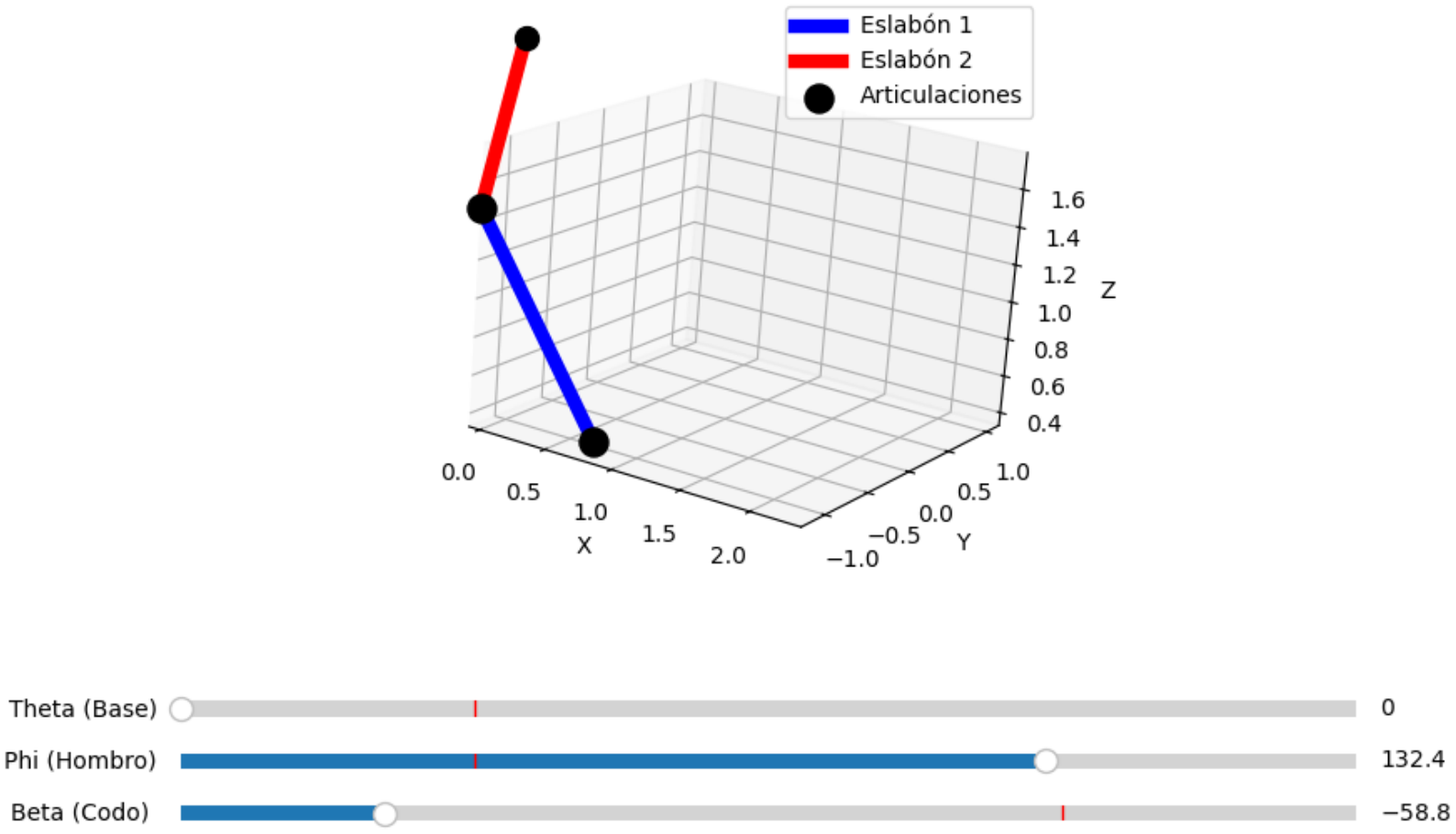

Figure 4: Finalmente se pone en una posición aleatoria.

Finalmente el sistema se empaqueto usando PyInstaller para garantizar la portabilidad del software en entornos Windows.

6 Conclusiones

-La eficacia de el modelo matemático respecto a la representación visual es efectiva - El programa es optimo, no presenta tirones al ejecutarse continuamente. -como posibles mejoras esta la opción de agregar cuantos eslabones se quieran, o usar cinemática inversa y aplicar colisiones, las posibilidades son infinitas.

7 Repositorio

Aquí esta el repositorio donde esta el ejecutable de este proyecto, ademas de el código fuente y demás proyectos que tengo alojados allí.

Repositorio: https://github.com/SienceNova/Proyectos.git